

A. VPC and Networking

1. What is a VPC and what are its components?

A VPC is your isolated virtual network in an AWS Region, spanning all AZs. Its components are:

- subnets (each in one AZ)
- route tables
- Internet Gateway (IGW)
- NAT Gateway
- security groups and NACLs
- VPC endpoints
- peering, Transit Gateway, VPN/Direct Connect
- DHCP options and Flow Logs

AWS reserves 5 IPs per subnet, and the CIDR can be /16 to /28.

2. What is a subnet, and what is the difference between public and private subnets? How do you configure each?

A subnet is a CIDR slice of the VPC that lives in exactly one AZ. The route table decides public vs private.

- **Public:** route 0.0.0.0/0 goes to an IGW, and instances need a public or Elastic IP (enable auto-assign public IP). Host ALBs, NAT Gateways and bastions here.
- **Private:** no IGW route. Outbound goes via 0.0.0.0/0 to a NAT Gateway. Host app servers and databases here.

3. What is an Internet Gateway? How do you implement it and attach it to a route table?

An IGW is a horizontally scaled, free, regional gateway that gives two-way internet access. Steps:

1. Create the IGW and attach it to the VPC.
2. Add the route 0.0.0.0/0 to the IGW in the public subnet's route table.
3. Make sure the instance has a public/Elastic IP and the SG/NACL allow the traffic.

```
aws ec2 create-internet-gateway
```

```
aws ec2 attach-internet-gateway --internet-gateway-id igw-x --vpc-id vpc-x
```

```
aws ec2 create-route --route-table-id rtb-x --destination-cidr-block 0.0.0.0/0 --gateway-id igw-x
```

4. What is a NAT Gateway, why do we use it, how does it work?

It gives private-subnet instances outbound-only internet access (patches, external APIs) while blocking inbound connections initiated from outside. It lives in a public subnet with an Elastic IP. Private route table: 0.0.0.0/0 goes to the NAT GW. The NAT GW translates the private source IP to its Elastic IP, sends the traffic out via the IGW, and maps replies back. It is zonal, so use one per AZ for HA. It is billed per hour plus per GB.

5. IGW vs NAT Gateway? NAT Gateway vs NAT Instance?

- **IGW:** two-way, free, for public resources. **NAT GW:** outbound-only, paid, for private resources.
- **NAT Gateway:** managed, scales to about 100 Gbps, highly available within its AZ, no patching.
- **NAT Instance:** an EC2 you manage. You must disable source/dest check, size it yourself, and handle patching and failover. It is cheaper at low traffic, and can double as a bastion or do port forwarding.

6. A private instance can't use a NAT Gateway. How else can it get packages or reach AWS services without the internet?

- **VPC endpoints:** S3 gateway endpoint (Amazon Linux repos are served from S3), plus interface endpoints for SSM, ECR, Secrets Manager, CloudWatch and STS.
- **Internal mirror:** a Nexus/Artifactory/yum mirror, or a Systems Manager Patch Manager baseline.
- **NAT instance or proxy** in a public subnet.
- **Centralised egress VPC** through a Transit Gateway with inspection.
- IPv6 workloads can use an egress-only internet gateway.

7. Interface VPC endpoint vs Gateway VPC endpoint?

- **Gateway:** only S3 and DynamoDB, free, works as a route-table entry (prefix list), and is regional and not reachable from on-prem.
- **Interface (PrivateLink):** an ENI with a private IP in your subnet, for most other services, billed hourly plus data, has SGs, and is reachable over VPN/DX.

8. How does routing work in a VPC? What is a route table, and how do you troubleshoot routing?

Every subnet is associated with one route table (the main one by default). Every table has an implicit local route for VPC CIDR traffic. The **most specific (longest prefix)**

route wins. Targets include IGW, NAT, TGW, peering, VPN, endpoints and ENIs.

Troubleshooting:

1. Check which route table the subnet is actually associated with.
2. Look for blackhole routes (a deleted target).
3. Check the return path, the SG/NACL, and the TGW route tables.
4. Run **VPC Reachability Analyzer**.
5. Check Flow Logs for REJECT.

9. Security Group vs NACL?

	SG	NACL
Level	ENI/instance	Subnet
State	Stateful	Stateless (open ephemeral ports 1024-65535 for returns)
Rules	Allow only, all evaluated	Allow + deny, evaluated in number order
Default	No inbound, all outbound	Default NACL allows all

Use SGs for normal control, including SG-to-SG references. Use NACLs to block specific IP ranges.

10. How does VPC peering work, what are its limitations, and how do you connect two VPCs?

Peering is a private one-to-one link using AWS's network. Steps:

1. The requester sends a request and the acceptor accepts it (cross-account and cross-region both work).
2. Add routes in both VPCs' route tables to the peer CIDR.
3. Update the SGs/NACLs.

Limitations: **non-transitive**, no overlapping CIDRs, no edge-to-edge routing (you can't use the peer's IGW, NAT or VPN), and it gets messy as a full mesh. For many VPCs use Transit Gateway. To expose only one service, use PrivateLink.

11. How does Transit Gateway work, how do you configure it, and what do you update in each VPC? Do you configure CIDRs on the attachment?

TGW is a regional hub-and-spoke router. Steps:

1. Create the TGW.

2. Create a **VPC attachment** for each VPC, choosing one subnet per AZ.
3. In the **TGW route table**, associate the attachments and enable route propagation (or add static routes).
4. In **each VPC's route table**, add routes to the other VPC CIDRs (or a summary like 10.0.0.0/8) with the TGW as target.
5. Update SGs/NACLs.

No, you don't enter a CIDR on the attachment. The attachment only picks subnets. CIDRs reach the TGW route table through propagation or static routes. Use separate TGW route tables to segment prod from non-prod. It also connects VPN/DX and supports inter-region peering and multicast.

12. How do you troubleshoot communication between (a) two instances in different subnets and (b) two VPCs?

(a) Check in this order:

1. SG on the destination (inbound from source IP/SG, correct port).
2. NACLs on both subnets, both directions.
3. The local route.
4. The OS firewall (iptables/firewalld), and whether the app is listening (ss -tlnp).
5. ICMP allowed if you are testing with ping.
6. Reachability Analyzer and Flow Logs.

(b) Check:

1. Peering/TGW status (active, accepted) and TGW association/propagation.
2. Routes on both sides.
3. Overlapping CIDRs.
4. SG/NACL referencing the correct CIDR.
5. DNS resolution across the peering (enable the DNS resolution option).
6. Cross-account/region specifics.

13. What are the dependencies for an Elastic IP / an IP-based resource?

An EIP is a regional static public IPv4 address. To be useful it must be associated with an instance, ENI or NAT Gateway inside a VPC that has an **IGW attached**, and the subnet needs a route to the IGW, with SG/NACL allowing the traffic. An unassociated EIP still costs money (all public IPv4 addresses are now charged).

14. What is Route 53? Is it mandatory here? How do you use a GoDaddy domain with it?

Route 53 is AWS's DNS, domain registration and health-check service. It is not mandatory, because you can keep DNS at GoDaddy and just add records pointing to your ALB. But Route 53 gives Alias records (free, at the zone apex), health checks, failover/weighted/latency routing, private hosted zones and tight AWS integration. To use a GoDaddy domain, there are two options:

1. **Keep the registrar at GoDaddy:** create a public hosted zone in Route 53, then copy its 4 NS records into GoDaddy's "custom nameservers" setting.
2. **Transfer registration** to Route 53 (unlock the domain, get the auth code).

You can also delegate only a subdomain by creating an NS record for it at GoDaddy.

15. What is a WAF? What would an "application access firewall" be?

AWS WAF is a Layer-7 firewall for HTTP(S). It uses web ACLs with managed and custom rules to block SQLi, XSS, bad bots and bad IPs, and to rate-limit. It attaches to CloudFront, ALB, API Gateway and AppSync. "Application access firewall" is not a standard AWS term, so ask the interviewer to clarify. The closest candidates are AWS Network Firewall (VPC-level, L3-L7 inspection), Security Groups, Firewall Manager (central policy), and AWS Verified Access (zero-trust access to apps without a VPN).

16. What are VPC Flow Logs? How do you find which IPs are hitting your VPC, and filter one IP in CloudWatch or in S3?

Flow Logs record IP traffic metadata (source, destination, ports, protocol, bytes, ACCEPT/REJECT) for a VPC, subnet or ENI, delivered to CloudWatch Logs or S3. They do not include payload.

- **CloudWatch Logs Insights:**

```
fields @timestamp, srcAddr, dstAddr, dstPort, action
```

```
| filter srcAddr = "203.0.113.10"
```

```
| sort @timestamp desc | limit 50
```

Aggregate top talkers with `stats count() by srcAddr | sort count desc`. The console filter pattern also works: "203.0.113.10".

- **In S3:** logs are gzipped, so create an **Athena** table over the prefix and run `SELECT * FROM vpc_flow_logs WHERE srcaddr='203.0.113.10'`. For a quick check, download the files and use `zgrep`.

17. The web application is unreachable but the EC2 instance is healthy. What are the major causes?

Status checks only prove the host and OS are alive, not the app. Check:

1. App process down or not listening on the expected port.
2. SG/NACL blocking.
3. Missing route or IGW, or no public IP.
4. DNS record wrong or TTL stale.
5. ALB target unhealthy (wrong health-check path or port), or listener or rule misconfigured.
6. WAF or CloudFront blocking.
7. Expired or invalid TLS certificate.
8. Disk full or out of memory so the app hangs.
9. OS firewall.

18. Given an empty AWS account, how do you create a VPC and everything needed to expose an EC2 service to the internet?

Services involved:

1. VPC with a CIDR plan, and subnets (public/private across 2 AZs).
2. IGW plus a public route table.
3. Security Group (80/443, 22 restricted or use SSM).
4. EC2 with an AMI, key pair or SSM role, and user data to install the app.
5. Optionally an Elastic IP.
6. Better: an ALB in public subnets with the app in private subnets, an ACM certificate, and a Route 53 record.
7. Optionally NAT GW, WAF, Flow Logs, and CloudWatch.

B. EC2, Storage and Backup

19. How do you connect to an EC2 instance? Which authentication does it use? Is a password needed?

```
chmod 400 key.pem
```

```
ssh -i key.pem ec2-user@<public-ip-or-dns>
```

The default user depends on the AMI: ec2-user (Amazon Linux/RHEL), ubuntu, admin (Debian). Authentication is by **key pair** (public key on the instance, private key on your machine). Password login is disabled by default. Alternatives are **SSM Session**

Manager (aws ssm start-session --target i-xxxx, no port 22 or keys needed) and EC2 Instance Connect.

20. How do you troubleshoot an EC2 instance that can't reach the internet, or that you can't SSH into?

No internet: check the SG outbound rules, the NACL, the route table (IGW for public, NAT for private), the public IP, NAT state and subnet, and DNS.

No SSH:

1. SG inbound 22 from your IP.
2. NACL.
3. The instance has a public IP and an IGW route.
4. Key pair/permissions (chmod 400) and the correct username.
5. Status checks, and the console output or screenshot.
6. Disk full.
7. sshd running (use SSM or the EC2 Serial Console).

21. What is EBS? EBS vs instance store? How do you attach an EBS volume?

EBS is network-attached block storage in one AZ, with snapshots and online resize.

Instance store is physical local disk: very fast, but **data is lost on stop, terminate or hardware failure**. Attaching:

```
aws ec2 create-volume --availability-zone ap-south-1a --size 20 --volume-type gp3
```

```
aws ec2 attach-volume --volume-id vol-x --instance-id i-x --device /dev/xvdf
```

on the instance

```
lsblk; sudo mkfs -t xfs /dev/xvdf; sudo mkdir /data; sudo mount /dev/xvdf /data
```

persist: add the UUID to /etc/fstab

The volume must be in the same AZ as the instance.

22. How do you attach EFS/NFS to an EC2 instance?

1. Create the EFS file system and mount targets in each AZ's subnet.
2. Make the mount-target SG allow **TCP 2049** from the instance SG.
3. On the instance, sudo yum install -y amazon-efs-utils, then sudo mount -t efs fs-xxxx:/mnt/efs (or an NFSv4.1 mount), and add it to /etc/fstab.

23. What happens to the instance and its EBS volumes on stop vs terminate?

- **Stop:** the instance shuts down, RAM and instance store are lost, EBS volumes persist (and are still billed), the public IP changes unless you use an EIP, and you can start it again (possibly on a different host).
- **Terminate:** the instance is deleted permanently. The root volume is deleted by default (DeleteOnTermination=true), extra volumes persist by default, and the EIP is disassociated. Enable termination protection for critical instances.

24. S3 vs EBS?

S3 is object storage accessed over an API/HTTP, with virtually unlimited capacity, 11 nines durability, multi-AZ in a Region, and cheap. EBS is block storage attached to one instance (one AZ) with low latency, used for boot disks and databases. Use S3 for files, backups, logs and static content, and EBS when you need a filesystem or disk.

25. What is S3 versioning and lifecycle policy? How do you handle accidental deletion, backup and retention?

- **Versioning** keeps every version of an object. A delete only adds a delete marker, so you can restore by removing the marker or fetching an older version. It protects against overwrites and deletes.
- **Lifecycle rules** automate transitions (Standard, then Standard-IA, then Glacier) and expiry, including noncurrent versions and incomplete multipart uploads.

Accidental deletion protection: versioning + MFA Delete, Object Lock, bucket policy denying s3:DeleteObject* for most users, and CRR to another bucket or Region.

Backup and retention: use lifecycle rules for retention periods, AWS Backup for S3, replication, and Glacier Vault/Object Lock for compliance.

26. How do you back up AWS services? Can it run from a shell script? Where do backup logs go?

- **AWS Backup:** central plans, vaults and retention across EC2, EBS, RDS, EFS, DynamoDB, S3 and more, assigned by tag, with cross-account/region copy and Vault Lock.
- **Native:** EBS snapshots (with Data Lifecycle Manager), RDS automated backups and snapshots, S3 versioning/replication.

Yes, a shell script works using the CLI, for example `aws ec2 create-snapshot --volume-id vol-x`, `aws rds create-db-snapshot ...` or `aws backup start-backup-job ....` Schedule it with cron, or better with EventBridge Scheduler + Lambda/SSM.

Logs: there is no traditional log file. Job status and history are in the AWS Backup console/API, completion events go to EventBridge/SNS, and API calls are in CloudTrail. AWS Backup Audit Manager can write reports to S3. For a script, redirect its output to a file or CloudWatch Logs.

27. Explain data storage on AWS and S3 in general.

S3 is object storage, EBS is block, EFS/FSx is file, RDS/Aurora is relational, DynamoDB is NoSQL, ElastiCache is in-memory, and Redshift is a warehouse. S3 concepts are buckets, keys, storage classes, versioning, lifecycle, replication, encryption (SSE-S3/KMS), strong read-after-write consistency, presigned URLs, and event notifications to Lambda/SQS/SNS.

C. IAM and Security

28. What is an IAM role, why do we use it, and how does it differ from a user?

A user has long-term credentials (password or access keys). A **role** has no permanent credentials and is **assumed** temporarily through STS by EC2, Lambda, pods, other accounts or federated identities. We use roles to avoid hard-coded keys, get auto-rotating temporary credentials, and give services and cross-account callers scoped access.

29. Walk me through granting a user access to an S3 bucket (full process).

1. Decide who needs it (IAM Identity Center user or role is preferred over a long-term IAM user), and exactly what access (read-only, a specific prefix).
2. Attach an **identity policy**: s3:ListBucket on arn:aws:s3:::bucket (with a prefix condition if needed) and s3:GetObject/PutObject on arn:aws:s3:::bucket/prefix/*.
3. If the bucket uses SSE-KMS, also grant kms:Decrypt/GenerateDataKey and check the key policy.
4. Check for blockers: bucket policy denies, SCPs, Block Public Access (irrelevant for IAM principals), and permission boundaries.
5. Require MFA for sensitive buckets.
6. Test with the IAM Policy Simulator or by assuming the role, and audit through CloudTrail.

30. How do you set up cross-account S3 access securely (App in Account A, bucket in Account B)?

Two common patterns:

- **Role assumption (recommended)**: in Account B create a role with a trust policy for Account A's role and an S3 permissions policy. The app's role in Account A needs sts:AssumeRole on it. The app assumes it and gets temporary credentials.
- **Bucket policy**: Account B's bucket policy allows Account A's role directly, and Account A's role policy also allows the S3 actions. Both sides must allow.

json

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::<AccountA>:role/app-role"
  },
  "Action": [
    "s3:GetObject",
    "s3:PutObject"
  ],
  "Resource": "arn:aws:s3:::bucket-b/*"
}
```

Also handle KMS key policy sharing, set ObjectOwnership to bucket-owner-enforced, and optionally restrict by aws:PrincipalOrgID.

31. How do you securely access S3 from EC2, and how do you secure a bucket?

Access: attach an **instance profile (IAM role)**, never access keys, and use a Gateway endpoint so traffic stays private. Securing the bucket: Block Public Access, a TLS-only policy (aws:SecureTransport), least-privilege policies, encryption (SSE-KMS), versioning/Object Lock, logging, Access Analyzer/Macie, VPC endpoint policies, and ACLs disabled.

32. How do you secure secrets in AWS? Which services store them? Secrets Manager vs Parameter Store?

Never store secrets in code, Git, images or plain env files. Store them in:

- **Secrets Manager:** automatic rotation (RDS etc.), versioning, cross-region replication, resource policies, KMS-encrypted, paid per secret.
- **Parameter Store (SecureString):** free standard tier, good for config and simple secrets, no native rotation.

Also use KMS, Sealed Secrets / External Secrets Operator on Kubernetes, and gitleaks/git-secrets scanning. Fetch secrets at runtime with IAM roles (IRSA for pods) and audit access in CloudTrail.

33. How do you implement least privilege and secure AWS credentials? How do you stop developers reading production secrets?

- **Least privilege:** start minimal, use IAM Access Analyzer policy generation and last-accessed data, scope by resource and conditions (tags, source VPC, MFA), use permission boundaries and SCPs, use groups/roles through Identity Center, and review regularly.
- **Credentials:** no root access keys, MFA, SSO with short-lived credentials, roles instead of keys, OIDC for CI, rotate or delete unused keys.
- **Prod secrets:** separate prod account, secret resource policies and KMS key policies allowing only the prod roles, an SCP/explicit deny for non-prod roles, break-glass access with approval, and CloudTrail alerts on GetSecretValue.

34. How do you implement good security policies on AWS overall?

1. **Identity:** SSO + MFA, roles, least privilege, SCPs.
2. **Network:** private subnets, SG/NACL discipline, endpoints, WAF/Shield, no public DB.
3. **Data:** encryption at rest (KMS) and in transit (TLS), S3 Block Public Access, backups.
4. **Detection:** CloudTrail (org-wide), Config, GuardDuty, Security Hub, Inspector, Macie, Flow Logs.
5. **Pipeline security:** image/IaC/secret scanning, signed artifacts.
6. **Governance:** multi-account, tagging, policy as code.
7. **Response:** incident runbooks and automated remediation.

35. AWS Config vs CloudTrail?

CloudTrail records **who did what** (API calls: user, time, source IP). Config records **what the resource looks like and how it changed**, and evaluates compliance rules.

Example: CloudTrail says "Bob opened port 22 at 10:05"; Config says "the SG is now non-compliant with the rule 'no 0.0.0.0/0 on SSH'".

36. How do you implement governance and enforce policies across multiple AWS accounts? How do you implement policy-as-code?

- **Governance:** AWS Organizations + Control Tower (guardrails, Account Factory), SCPs, tag policies, backup policies, Config aggregator and conformance packs, delegated Security Hub/GuardDuty, and IAM Identity Center.
- **Policy as code:** write rules as code and run them in CI and at runtime.
 - Terraform/CloudFormation: Checkov, tfsec, cfn-guard, OPA/Conftest, Sentinel.
 - Kubernetes: OPA Gatekeeper or Kyverno.
 - AWS: SCPs and Config custom rules (Lambda or Guard), all version-controlled and deployed through pipelines.

37. What security scanning tools do you know, and how do you scan Docker images at build time and in the registry?

Tools: Trivy, Grype, Clair, Snyk, Docker Scout, ECR scanning (Inspector), SonarQube (SAST), OWASP Dependency-Check, Checkov (IaC), gitleaks (secrets), OWASP ZAP (DAST).

- **Build time:** a Trivy stage in the pipeline (trivy image --severity HIGH,CRITICAL --exit-code 1 app:tag) fails the build on findings.
- **Registry:** ECR scan-on-push (enhanced scanning with Inspector gives continuous rescans as new CVEs appear), plus an admission policy in the cluster that blocks unscanned or unsigned images.

38. What is "signing" here: email signing and Helm chart signing? Which tools sign Helm charts?

Signing proves **integrity and authenticity**: the artifact is unchanged and comes from who it claims. Email signing uses S/MIME or PGP so the recipient can verify the sender (DKIM does it at the domain level). For Helm, helm package --sign --key '<name>' --keyring ~/.gnupg/secring.gpg mychart creates a .prov provenance file using **GPG**, and helm verify or helm install --verify checks it. For OCI charts and images, use **Cosign (Sigstore)** or Notation.

39. How do you handle EKS cluster authentication and store secrets securely?

- **Authentication:** IAM identity, then aws eks update-kubeconfig, which runs aws eks get-token. Authorisation comes from Kubernetes RBAC, with IAM principals mapped through **EKS access entries** (or the older aws-auth ConfigMap). SSO users use OIDC/Identity Center roles. Pods use IRSA or Pod Identity.
- **Secrets:** AWS Secrets Manager with External Secrets Operator or the Secrets Store CSI driver, Sealed Secrets for GitOps, KMS envelope encryption for etcd secrets, RBAC limiting get secrets, and no plaintext in Git or values files (helm-secrets/SOPS if needed).

D. Compute, Containers and Load Balancing

40. EC2 vs ECS vs EKS, and when do you use each? EKS vs ECS?

- **EC2:** full control, legacy or stateful apps, special hardware or licensing.
- **ECS:** AWS-native container orchestration, simple, deep IAM/ALB integration, lower learning curve.
- **EKS:** managed Kubernetes, portable and ecosystem-rich (Helm, ArgoCD, operators), more operational complexity and a control-plane fee.

Choose EKS if you need Kubernetes tooling or multi-cloud portability, ECS for simple AWS-only container workloads, and EC2 when containers don't fit.

41. How did you set up ECS backed by EC2 instances?

1. Create an ECS cluster and a launch template using the ECS-optimized AMI, with an instance profile (ecsInstanceRole) and user data echo ECS_CLUSTER=<name> >> /etc/ecs/ecs.config.
2. Put it in an ASG across AZs, and create a **capacity provider** with managed scaling.
3. Create a **task definition** (image, CPU/memory, ports, task role, execution role, logging).
4. Create a **service** (desired count, awsvpc or bridge networking) behind an ALB target group.

Compared with Fargate, you manage the instances (patching, scaling the capacity).

42. What are the node group types in EKS and which did you use?

- **Managed node groups:** AWS manages the ASG/AMI lifecycle and upgrades (the usual choice).
- **Self-managed nodes:** your own ASG/AMI, for custom needs.
- **Fargate profiles:** serverless pods.
- **Karpenter-provisioned nodes** for fast, cost-efficient scaling, and **EKS Auto Mode** (newer, fully managed compute).

(adapt) "We used managed node groups, with On-Demand for critical workloads and Spot for stateless ones." If asked "which node types did you deploy", also name the EC2 families (general purpose m5/m6i, compute c5, memory r5, Graviton).

43. What are the prerequisites for an EKS cluster with two worker nodes and some pods?

1. A VPC with subnets in at least 2 AZs, tagged for load balancers, with internet or endpoint access for nodes.
2. IAM: a cluster role (AmazonEKSClusterPolicy) and a node role (AmazonEKSWorkerNodePolicy, AmazonEKS_CNI_Policy, AmazonEC2ContainerRegistryReadOnly).
3. Tools: aws cli, kubectl, eksctl or Terraform.
4. Create the cluster, then a managed node group with desired size 2, with add-ons (VPC CNI, CoreDNS, kube-proxy).
5. Update kubeconfig, then deploy pods with a Deployment and Service.

```
eksctl create cluster --name demo --nodes 2 --node-type t3.medium
```

```
aws eks update-kubeconfig --name demo
```

44. With two or more pods, how do you load balance? Are you certain ALB is right?

ALB vs NLB vs CLB?

Expose pods with a Service, and use an **Ingress** handled by the AWS Load Balancer Controller to create an **ALB** with target-type: ip, or a Service of type LoadBalancer to create an **NLB**. ALB is right when you need HTTP/HTTPS, path/host routing, or WAF/OIDC auth. It's not right for non-HTTP TCP/UDP or when you need static IPs or the lowest latency, so use NLB then.

- **ALB (L7):** content-based routing, redirects, WebSocket/gRPC, Lambda targets.
- **NLB (L4):** TCP/UDP/TLS, static or Elastic IPs, preserves source IP, extreme throughput.
- **CLB:** legacy, basic L4/L7, no path routing, not recommended for new work.

An ALB works through listeners, then rules, then target groups with health checks. An NLB forwards flows to healthy targets using flow-hash routing.

45. How does Auto Scaling work, including with an ALB? How do you configure, verify and troubleshoot it?

An ASG keeps instances between min/max/desired across AZs using a Launch Template. Policies: target tracking, step, scheduled, predictive. With an ALB, register the ASG with the target group: new instances are registered automatically, and the ELB health check can replace unhealthy ones.

- **Configure:** a Launch Template (AMI, type, SG, IAM profile, user data), subnets in multiple AZs, capacity limits, the target group, a scaling policy, and a health check grace period.
- **Health checks:** EC2 type (instance status) vs ELB type (app-level, preferred behind an ALB).
- **Verify:** the ASG activity history, CloudWatch metrics (GroupInServiceInstances), the alarm state, and a load test.
- **Not scaling?** Max capacity reached, cooldown/warm-up, alarm in INSUFFICIENT_DATA or wrong metric, launch failures (AMI, quota, IAM, no capacity or IPs in subnet), suspended processes, or a wrong health check.

46. How do you implement high availability across multiple AZs?

Subnets in at least 2-3 AZs, an ALB (cross-zone), an ASG spread across AZs, one NAT per AZ, RDS/Aurora Multi-AZ, EFS or S3 (multi-AZ by design), stateless apps, and health checks with auto-healing. Test by terminating instances or simulating an AZ failure.

47. How do you create Lambda functions and push the artefact? Explain Lambda.

Lambda runs code in response to events with no servers to manage. You pay per request and duration, with a 15-minute limit. Triggers include API Gateway, S3, SQS, EventBridge and DynamoDB Streams. Create it via the console, CLI, SAM, Terraform or CloudFormation. Artefact options:

- Direct **zip upload** (about 50 MB zipped, 250 MB unzipped):
`aws lambda update-function-code --function-name f --zip-file fileb://function.zip`
- A zip in **S3** (`--s3-bucket ... --s3-key ...`) for larger packages.
- A **container image from ECR** (up to 10 GB).
- **Layers** for shared dependencies, plus versions and aliases for safe rollout.

Do it from CI/CD, not by hand.

48. Explain CloudFormation templates.

CloudFormation is AWS-native IaC: you describe resources in JSON/YAML and AWS creates them as a **stack**. Template sections: AWSTemplateFormatVersion, Parameters, Mappings, Conditions, Resources (the only required section), Outputs. Intrinsic functions: `!Ref`, `!GetAtt`, `!Sub`, `!ImportValue`. Features: change sets, drift detection, rollback on failure, nested stacks and StackSets for multi-account.

E. HA and Disaster Recovery

49. What are RPO and RTO? How do you design a highly available architecture, multi-AZ apps and DR?

RPO is how much data you can lose (time). RTO is how long you can be down.

- **HA design:** multi-AZ, no single points of failure, stateless tiers, managed Multi-AZ data stores, health checks and auto-healing.
- **DR strategy** (cheapest to fastest): backup and restore, pilot light, warm standby, multi-site active-active. Pick one by RTO/RPO and cost.

50. How do you design multi-zone failover for stateful services with near-zero RPO?

Use synchronous replication across AZs: RDS Multi-AZ or Aurora (6 copies across 3 AZs), EFS or S3 (multi-AZ), and quorum-based clustered stores. For cross-region near-zero RPO, use Aurora Global Database (lag typically under a second), DynamoDB Global Tables, and S3 replication with Replication Time Control. Keep the app stateless so only the data tier matters.

51. How do you test a DR plan without impacting production?

- Restore backups into an isolated account/VPC and validate data.
- Run game days in staging, and use AWS Fault Injection Service for chaos tests.
- Do a partial production test by shifting 1-5% of traffic with weighted DNS to the DR region.
- Do tabletop exercises and time the runbook, then record actual RTO/RPO and fix the gaps.

52. How do you handle DNS-level failures?

Use Route 53 (its four name servers span TLDs), health checks with failover records, short TTLs on critical records, and optionally a secondary DNS provider. Add client retries, cache resilience, and Route 53 Application Recovery Controller for controlled failover.

53. How do you detect and respond to partial regional outages?

- **Detect:** synthetic canaries, per-AZ and per-service CloudWatch metrics and composite alarms, the AWS Health Dashboard, and error-rate/latency SLO burn alerts.
- **Respond:** follow the runbook, shift traffic away from the impaired AZ (ALB/ARC zonal shift, ASG AZ rebalancing), fail over the data tier if needed, scale up the healthy capacity, communicate, and run a post-mortem.

F. Automation, Governance and Cost

54. How do you automate provisioning of secure environments for new projects, and manage network dependencies across environments?

Use account vending (Control Tower Account Factory or AFT) plus versioned Terraform baseline modules (VPC, logging, IAM roles, guardrails, Config/GuardDuty enabled), deployed through pipelines with policy checks. For networking, keep the network layer in its own stack/state, plan CIDRs centrally (VPC IPAM), attach VPCs to Transit Gateway, and pass IDs to other stacks via remote state outputs or SSM parameters. Never hard-code subnet IDs.

55. How do you implement and enforce tagging policies?

Define a standard (owner, environment, cost-center, app). Enforce it with Organizations **tag policies**, SCPs requiring `aws:RequestTag` on creation, Config rule `required-tags`, Terraform `default_tags` and Checkov in CI, and auto-remediation or notification for non-compliance. Activate cost-allocation tags for billing.

56. How do you detect and clean up orphaned or stale resources?

- **Detect:** unattached EBS volumes, unused EIPs, old snapshots/AMIs, idle load balancers and NAT gateways, stopped instances, and empty SGs, using Trusted Advisor, Cost Explorer, Config, Compute Optimizer and Resource Explorer, or scheduled Lambda scripts.
- **Clean up:** notify the tagged owner, wait a grace period, snapshot first, then delete automatically. Use TTL tags for sandbox accounts and tools like cloud-nuke there.

57. How did you do cost optimisation in AWS? How would you cut cost by 20%? Have you written cost automation?

1. **Visibility:** tags, Cost Explorer, Budgets, anomaly detection.
2. **Right-size** with Compute Optimizer and CloudWatch metrics, and move to Graviton.
3. **Pricing:** Savings Plans/RIs for the baseline, Spot for stateless and CI.
4. **Schedule** dev/test off outside working hours (typically a large saving).
5. **Storage:** S3 lifecycle and Intelligent-Tiering, gp2 to gp3, delete orphaned volumes and snapshots.
6. **Network:** endpoints to avoid NAT data charges, keep traffic in-AZ.
7. **Kubernetes:** Karpenter consolidation, right-sized requests/limits, HPA.

Automation examples (adapt): a Lambda that stops tagged dev instances at night, a script that reports unattached EBS volumes and old snapshots, and budget alerts to Slack.

58. How do you count running EC2 instances and attached EBS volumes across 50-60 accounts without logging into each?

- **Best:** an **AWS Config aggregator** (organization-wide) with advanced queries: `SELECT COUNT(*) WHERE resourceType='AWS::EC2::Instance' AND configuration.state.name='running'` and similarly for `AWS::EC2::Volume` filtered on attachment state. AWS Resource Explorer and Systems Manager Explorer also work.
- **Script:** list accounts with Organizations, assume `OrganizationAccountAccessRole` in each, loop over Regions and run `describe-instances / describe-volumes`:

bash

```
for acc in $(aws organizations list-accounts --query 'Accounts[].Id' --output text); do
```

assume role -> export temp creds, then:

```
aws ec2 describe-instances --filters Name=instance-state-name,Values=running \
--query 'length(Reservations[].Instances[])'
```

done

G. CI/CD, Jenkins and Git

59. Explain your CI/CD workflow and pipeline type. How do you define and invoke pipelines in Jenkins? (adapt)

Developer pushes to a feature branch, a PR is raised, and the **Multibranch Pipeline** picks up the Jenkinsfile in the repo. Typical stages: checkout, build, unit tests, SonarQube + quality gate, Docker build, Trivy scan, push to ECR, update the image tag in the Helm/GitOps repo, then ArgoCD syncs to the cluster, then smoke tests and Slack notification, with a manual approval gate for prod.

- **Defined** in a Jenkinsfile (Pipeline as Code) in the repo.
- **Invoked** by a webhook (preferred), SCM polling, a cron trigger, manually, or an upstream job.

60. What are Jenkins shared libraries, how are they written and declared? (asked twice, merged)

They are reusable Groovy code shared across many Jenkinsfiles, so pipelines stay DRY and standardised. Repo structure:

```
vars/  buildImage.groovy, deployHelm.groovy  (global steps, define call())
src/   org/company/Utils.groovy              (classes)
resources/                                (static files, templates)
```

- **Declared:** Manage Jenkins, System, Global Pipeline Libraries (name, default version, Git source), or at folder level.
- **Used** in the Jenkinsfile with `@Library('my-lib@main')` __, then call `buildImage(name: 'app')`.

61. What kinds of apps do you deploy through Jenkins, and with which tools?

(adapt)

Containerised microservices (Java/Spring, Node, Python) deployed to Kubernetes with Helm and ArgoCD, plus ECS services and Lambda functions. Tools: Docker, Helm, kubectl, Terraform, AWS CLI, ArgoCD.

62. The Jenkins pipeline runs but no build happens. What could cause it?

- **Trigger issues:** the webhook isn't firing (wrong URL, secret, firewall, Jenkins not reachable), the branch or SCM filters exclude it, the multibranch scan didn't run.
- **Agents:** no executor free, an agent label mismatch, or the node is offline, so the job is stuck in the queue.
- **Pipeline logic:** when conditions skipping every stage, a missing or misnamed Jenkinsfile, or "no changes" logic.
- **Environment:** credentials failing, disk full on the controller or agent, the job disabled.

63. What is a webhook and how is it used in CI/CD?

A webhook is an HTTP callback: when an event happens (push, PR), the Git server POSTs to a URL, instantly triggering the pipeline, instead of Jenkins polling the repo. For Jenkins and GitHub the URL is `https://<jenkins>/github-webhook/`.

64. Given a fresh Jenkins, how do you connect it to GitHub? Is webhook communication available out of the box?

Ways: the **GitHub plugin + webhook**, **GitHub Branch Source** (Multibranch/Organization folders) with a GitHub App or PAT credential, Generic Webhook Trigger, SCM polling, or SSH deploy keys. Webhooks are **not fully out of the box**: you need to install the plugins, add credentials, make Jenkins reachable from GitHub (public URL, reverse proxy or tunnel), and create the webhook in GitHub.

65. What kinds of stages exist, and can more than two run at once?

Build, test, scan, package, publish, deploy, approval (input), and notify, with post actions for cleanup. **Yes, any number of stages can run in parallel** using `parallel { stage('A'){...} stage('B'){...} }`, limited only by available executors.

66. Have you written Groovy? Declarative vs scripted pipeline?

- **Declarative:** a structured pipeline `{ agent, stages, steps, post, when }` syntax, validated up front and easier to read. Recommended.
- **Scripted:** a node `{ ... }` block with full Groovy, so it is flexible but more complex and error-prone.

You can embed a script `{ }` block in declarative for logic. (Be honest on depth: "I write Jenkinsfiles and shared-library steps; I'm comfortable with Groovy for pipeline logic.")

67. How do you integrate SonarQube into the pipeline?

Install the SonarQube Scanner plugin, add the server URL and token in Jenkins, then:

groovy

```
stage('Sonar') { steps { withSonarQubeEnv('sonar') { sh 'sonar-scanner' } } }
```

```
stage('Quality Gate') { steps { timeout(time:5, unit:'MINUTES') { waitForQualityGate  
abortPipeline: true } } }
```

Use a sonar-project.properties file, and configure a SonarQube webhook back to Jenkins so the gate result is returned.

68. What branching strategy do you follow? How do you keep merges from breaking release, and what do you do when a bug hits production? (adapt)

Feature branches off develop/main (GitFlow or trunk-based), merged by PR only.

Protection: required reviews, CODEOWNERS, required CI checks (tests, Sonar, scans), no direct pushes, squash/rebase merges. For a production bug: roll back first if impact is high, then create a **hotfix branch** from the release tag/main, fix, test, merge to main and back to develop, tag, deploy, and write an RCA.

69. How do you enforce full quality checks during deployment?

Quality gates in the pipeline: unit tests with coverage threshold, SonarQube gate, dependency and secret scans, image scan (Trivy), IaC scan (Checkov), integration/smoke tests post-deploy, manual approval for prod, and automatic rollback on failed health checks.

70. How do you migrate a Git repo from GitHub to GitLab preserving full history?

```
bash
```

```
git clone --mirror https://github.com/org/repo.git
```

```
cd repo.git
```

```
git remote set-url --push origin https://gitlab.com/org/repo.git
```

```
git push --mirror
```

Create the empty target project first. --mirror carries all branches and tags. If you use Git LFS, run git lfs fetch --all and git lfs push --all. Afterwards, update CI config, webhooks, protected branches and permissions. Issues and PRs are not part of Git, so use GitLab's built-in GitHub importer for those.

71. git fetch vs git pull, and git cherry-pick?

git fetch downloads remote changes without touching your working branch (safe, lets you inspect first). git pull = fetch + merge (or rebase with --rebase). git cherry-pick <sha> applies one specific commit onto the current branch, useful for hotfix backports. Use -x to record the source commit, and resolve conflicts then git cherry-pick --continue.

72. How do you handle merge conflicts? Do you inspect the source or target branch history?

git status, open conflicted files, resolve the <<<<<< ===== >>>>>> markers, git add, then git commit (or --continue). To decide correctly, look at **both**: what the source

branch changed (git log target..source) and what the target changed since the common ancestor (git log source..target), or git log --merge -p for the conflicting commits. Understand the intent, then rebuild and test before pushing.

73. Which CI/CD tools do you use? (adapt)

Jenkins (Multibranch pipelines), GitHub Actions, and ArgoCD for GitOps delivery, plus Helm, Docker, Terraform, SonarQube and Trivy in the pipelines.

H. Docker, Kubernetes and Helm

74. Explain the high-level Kubernetes architecture, and what the control plane and worker nodes do.

- **Control plane:** kube-apiserver (front door), etcd (state store), kube-scheduler (places pods), controller-manager (reconciliation loops), cloud-controller-manager.
- **Worker nodes:** kubelet (runs pods), kube-proxy (service networking), and a container runtime (containerd).

On EKS, AWS runs the control plane and you run the nodes.

75. How do you create and manage clusters with Terraform?

Use the terraform-aws-modules/eks/aws module (or aws_eks_cluster + aws_eks_node_group): the VPC module first, then the cluster with IAM roles, OIDC provider, node groups and add-ons. Keep remote state (S3 with locking), one folder per environment, and use terraform plan in PRs. Manage the apps inside the cluster with Helm/ArgoCD rather than Terraform.

76. Which Kubernetes errors have you hit (CrashLoopBackOff, ImagePullBackOff) and how did you resolve them?

kubectl describe pod first, then logs.

- **CrashLoopBackOff:** kubectl logs --previous. Causes: app error, bad config or secret, failing liveness probe, OOMKilled (raise memory limits).
- **ImagePullBackOff:** wrong image or tag, no ECR permission (node role or pull secret), no route to the registry, or rate limits.
- **Pending:** insufficient resources, taints, PVC bound to another AZ, or no IPs.

77. What is the command to get a shell in a pod, and how do you define a Kubernetes object?

kubectl exec -it <pod> -n <ns> -c <container> -- /bin/sh

An object is defined in YAML with apiVersion, kind, metadata, spec, then applied with `kubectl apply -f file.yaml`. Generate a skeleton with `kubectl create deployment web --image=nginx --dry-run=client -o yaml`.

78. Explain the Helm chart folder structure and the deploy commands. What are Helm charts for?

Helm is the package manager for Kubernetes: it templates manifests, parameterises them with values, and versions releases (with rollback).

mychart/

Chart.yaml (name, version, appVersion, dependencies)

values.yaml (default values)

templates/ (deployment.yaml, service.yaml, ingress.yaml, _helpers.tpl, NOTES.txt, tests/)

charts/ (subcharts)

.helmignore

Commands: `helm lint`, `helm template`, `helm upgrade --install app ./mychart -f values-prod.yaml -n prod --atomic`, `helm list`, `helm history`, `helm rollback app 1`, `helm uninstall app`.

79. How do you achieve zero downtime during an EKS cluster upgrade?

1. Check for deprecated APIs and add-on compatibility.
2. Upgrade the control plane one minor version at a time.
3. Upgrade the add-ons (CNI, CoreDNS, kube-proxy).
4. Roll the node groups, respecting **PodDisruptionBudgets**, with at least 2 replicas spread across AZs and readiness probes. Or use blue/green node groups.
5. Test in a lower environment first.

80. What are the stages of a Docker image build, and why do both ENTRYPOINT and CMD exist?

A build runs the Dockerfile instructions top to bottom (FROM, COPY, RUN, ENV, EXPOSE, CMD), creating a cached **layer** for each. Multi-stage builds use a builder stage and a slim runtime stage to keep images small and secure.

- **ENTRYPOINT**: the fixed executable.
- **CMD**: the default arguments, easily overridden at docker run.

Together, ENTRYPOINT ["java","-jar","app.jar"] + CMD ["--spring.profiles.active=prod"] lets users change only the arguments.

81. How do you pass environment variables into docker build, and where do you store images?

Use ARG + docker build --build-arg VERSION=1.2 . for build-time values (ENV makes them persist at runtime). **Never pass secrets through ARG/ENV**, since they stay in image history. Use BuildKit secrets: docker build --secret id=npm,src=.npmrc .. Images are stored in a registry: ECR (AWS), GCR, or Docker Hub (adapt: say which you used).

82. Dockerfile for Tomcat on 8080: build it and run a container exposing 9090?

```
docker build -t tomcat-app .
```

```
docker run -d -p 9090:8080 --name tomcat-app tomcat-app
```

-p host:container, so host port 9090 maps to container port 8080. EXPOSE only documents the port and doesn't publish it.

83. How do you manage and connect to services (databases, EC2, EKS, ECS)? How do you connect to ECS?

- **DB connections in deployments:** credentials from Secrets Manager (via External Secrets) injected as env vars or mounted files, DB in private subnets with SG allowing only the app SG, RDS Proxy for pooling, and Flyway/Liquibase for migrations. For admin access, use SSM port forwarding or a bastion.
- **EC2:** SSH or aws ssm start-session.
- **EKS:** aws eks update-kubeconfig --name <cluster>, then kubectl.
- **ECS: ECS Exec:**

```
aws ecs execute-command --cluster c --task <task-id> --container app --interactive --command "/bin/sh"
```

It needs enableExecuteCommand on the service and SSM messages permissions on the task role.

84. What is the biggest issue you resolved in Kubernetes? (adapt, use STAR)

Pick one real incident. Example shape: pods were repeatedly OOMKilled during peak load, causing 5xx errors. You traced it with kubectl describe and Prometheus memory graphs, found a memory leak plus limits set too low, raised limits, tuned the JVM heap, added an HPA and an alert on memory saturation, and errors dropped to zero. Include the impact, your actions and the permanent fix.

85. What have you done on Linux and what installations have you performed? CPU cores? Cron? File size? (adapt)

Administration (users, permissions, services with systemctl, logs, disk and process troubleshooting) and installs using package managers (yum/dnf/apt), rpm/dpkg, tarballs and source builds.

```
nproc          # CPU cores (also: lscpu, grep -c ^processor /proc/cpuinfo)
```

```
crontab -e      # edit; crontab -l to list
```

```
# min hour dom mon dow  command
```

```
30 2 * * * /opt/backup.sh >> /var/log/backup.log 2>&1 # daily 02:30
```

```
du -sh /path/file # file size (also: ls -lh file, stat -c %s file)
```

```
find / -name "app.log" -exec ls -lh {} \; # find a file in a tree and show its size
```

A cron job is a scheduled task run by the cron daemon (backups, log rotation, cleanups).

86. Have you worked on Ansible? What have you done with Terraform on AWS?

(adapt)

- **Ansible** is agentless config management over SSH. It uses an inventory, YAML playbooks, roles and modules, and is idempotent (patching, installing packages, config files).
- **Terraform on AWS:** VPCs, subnets, EC2/ASG, EKS, RDS, S3, IAM roles, security groups, using reusable modules, remote state in S3 with locking, and a plan/apply pipeline.

87. Terraform vs CloudFormation?

Terraform is multi-cloud with HCL, you manage state, it has a rich module registry, and uses plan/apply. CloudFormation is AWS-only with AWS-managed state, native rollback, change sets, drift detection and StackSets. Terraform suits multi-cloud or broad provider use, CloudFormation suits AWS-only shops that want no state management.

88. Write Terraform for an EC2 instance with an EBS volume attached.

```
hcl
```

```
data "aws_ami" "al2023" {
```

```
    most_recent = true
```

```
    owners      = ["amazon"]
```

```
    filter { name = "name" values = ["al2023-ami-*-x86_64"] }
```



```
}
```

```
resource "aws_instance" "app" {  
  ami           = data.aws_ami.al2023.id  
  instance_type = "t3.micro"  
  subnet_id     = var.subnet_id  
  vpc_security_group_ids = [aws_security_group.app.id]  
  tags = { Name = "app" }  
}
```

```
resource "aws_ebs_volume" "data" {  
  availability_zone = aws_instance.app.availability_zone  
  size             = 20  
  type             = "gp3"  
  encrypted        = true  
}
```

```
resource "aws_volume_attachment" "data" {  
  device_name = "/dev/xvdf"  
  volume_id   = aws_ebs_volume.data.id  
  instance_id = aws_instance.app.id  
}
```

89. What is your Terraform file structure for a VPC and EKS? (adapt)

modules/

vpc/ (main.tf variables.tf outputs.tf)

eks/ (main.tf variables.tf outputs.tf)

envs/

dev|stage|prod/ (main.tf providers.tf backend.tf variables.tf terraform.tfvars)

Each environment calls the modules, has its own remote state, and passes the VPC outputs (VPC ID, subnet IDs) into the EKS module.

90. What do terraform init and terraform refresh do?

init prepares the working directory: it downloads providers and modules and configures the backend. refresh reads the real infrastructure and updates the **state file** to match, without changing any resources. Standalone refresh is deprecated; use terraform apply -refresh-only, and note that plan already refreshes by default.

91. What have you done with monitoring, and what are your top five technologies?

(adapt)

Monitoring: Prometheus and Grafana for cluster and app metrics, ELK/Fluentd for logs, CloudWatch and CloudTrail on AWS, with alerts routed to Slack or on-call. The focus is on SLIs (latency, errors, saturation) and actionable alerts.

Top five suggested for your profile: **Kubernetes, AWS, Docker/Helm, ArgoCD/GitOps with CI/CD (Jenkins/GitHub Actions), Terraform.** Choose ones you can talk about for 10 minutes.

J. General and HR

92. Explain the OSI model.

Seven layers, top to bottom (mnemonic: *All People Seem To Need Data Processing*):

Layer	Role	AWS/DevOps link
7 Application	HTTP, DNS, SSH	ALB, WAF, API Gateway
6 Presentation	TLS, encoding	TLS termination, ACM
5 Session	Session management	
4 Transport	TCP/UDP, ports	NLB, Security Groups
3 Network	IP, routing	VPC routing, NACL, TGW
2 Data Link	MAC, frames	ENI
1 Physical	Cables, signals	AWS data center

In practice you troubleshoot bottom-up: IP and route (L3), port and SG (L4), then the app (L7).

93. Explain how you carried out a cloud migration. (adapt)

1. **Assess:** inventory and dependencies, pick a 7 R's strategy per app (rehost, replatform, refactor...).

2. **Landing zone first:** accounts, networking, IAM, logging, guardrails.
3. **Migrate:** servers via Application Migration Service, databases via DMS (plus SCT if heterogeneous), files via DataSync.
4. **Test and cut over:** parallel run, low-TTL DNS switch, a rollback plan.
5. **Optimise:** right-size, adopt managed services, add monitoring, review cost and security.

Include your real example: what moved, what challenges came up, and the result.

94. What motivates you about this role? (adapt)

"I enjoy building platforms that make other engineers faster and more reliable. This role combines what I've been doing with AWS, Kubernetes, IaC and CI/CD with a chance to go deeper into architecture, scale and reliability. I'm motivated by automating repetitive work, improving deployment safety and cost, and learning from a senior team that works on real production-scale problems."